



Chapter 11:

Inheritance and Polymorphism

Part-I

11.1. Inheritance

Motivation

- ✨ In OOP, we often need to create new classes that share common properties and behaviors with existing classes.
- ✨ It causes code duplication if we create similar classes with the same properties and methods.
- ✨ This can lead to maintenance issues and makes the code harder to read and understand.
- ✨ Therefore, the concept of **inheritance** is introduced to solve this problem.

What is Inheritance?

- ✨ **Inheritance** is a mechanism in OOP that allows a new class (subclass or child class) to inherit properties and methods from an existing class (superclass or parent class).
- ✨ The subclass can add new properties and methods or override existing ones from the superclass.
- ✨ This creates a hierarchical relationship between classes, where the subclass is a specialized version of the superclass.
- ✨ Inheritance allows for code reuse, making it easier to maintain and extend software systems.

👉 Why Use Inheritance?

- ✨ **Code Reusability:** Avoids rewriting common properties and behaviors.
- ✨ **Maintainability:** Easier to manage and update code.
- ✨ **Extensibility:** New classes can be added with minimal changes to existing code.

Example: Geometric Objects

- ✨ **Common properties:** color, filled status, creation date.
- ✨ **Specialized classes:** Circle, Rectangle, Triangle.
- ✨ **General class:** GeometricObject to model all geometric objects.

11.2. Superclasses and Subclasses

👉 **Superclass (Parent Class) and Subclass (Child Class):**

✨ **Superclass** or **Parent Class** is the general class from which other classes inherit.

✨ Contains common properties and methods that can be shared with subclasses.

✨ **Subclass** or **Child Class** is a specialized version of the superclass.

✨ Inherits properties and methods from the superclass.

✨ Can add new properties and methods or override existing ones.

Note: The relationship between a superclass and subclass is often described as an "**is-a**" relationship.

👉 The `extends` Keyword

✨ The `extends` keyword is used to create a subclass from a superclass in Java.

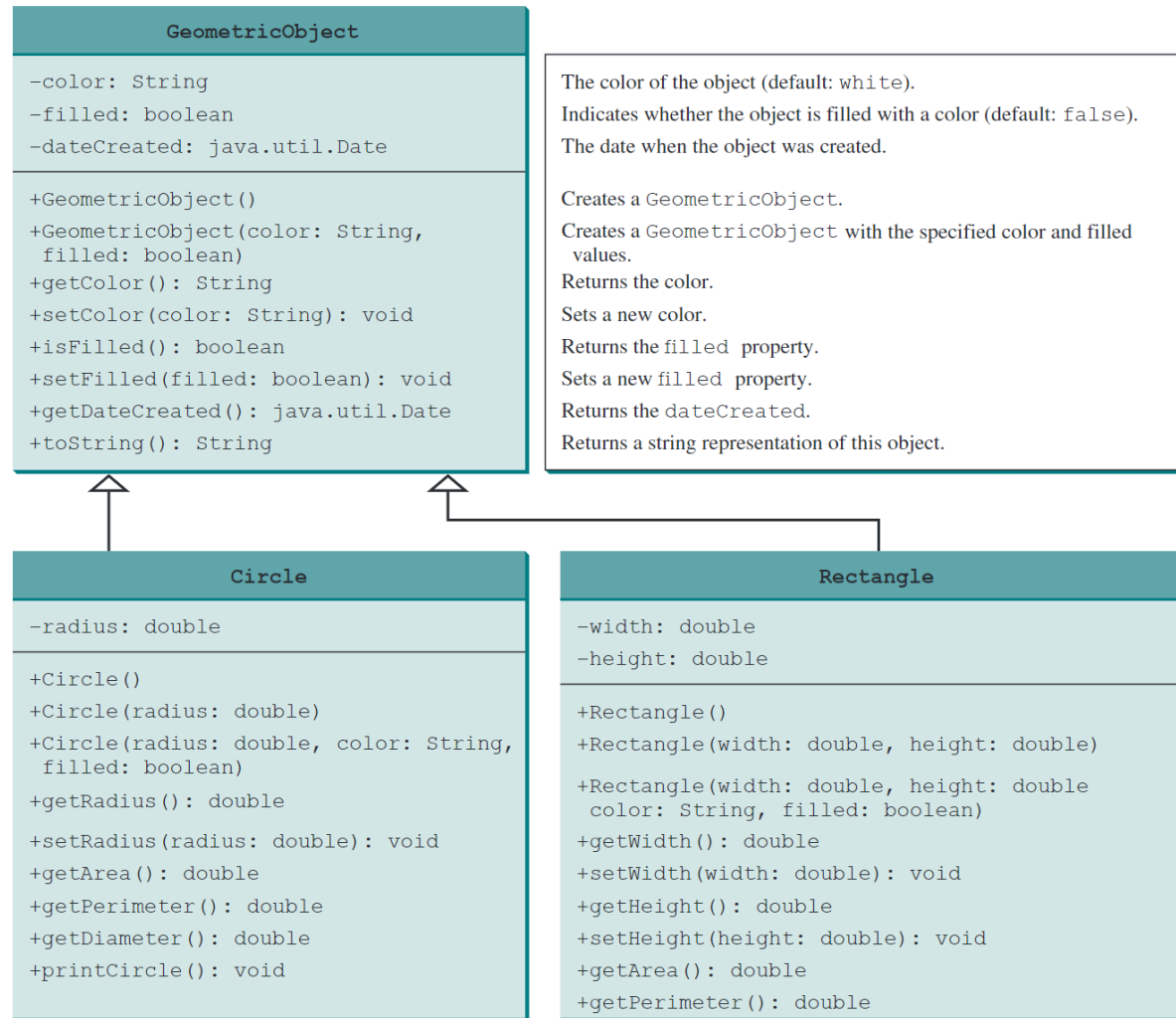
Syntax:

```
class SubclassName extends SuperclassName {  
    // Subclass properties and methods  
}
```

Explain:

✨ This establishes an inheritance relationship where `SubclassName` inherits from `SuperclassName`.

👉 Example of GeometricObject Class



👉 **Superclass: `GeometricObject`**

✨ **Attributes**

- ✨ `color` : The color of the geometric object.
- ✨ `filled` : Indicates whether the object is filled with color.
- ✨ `dateCreated` : The date the object was created.

✨ **Methods**

- ✨ `getColor()` : Returns the color of the object.
- ✨ `setColor(String color)` : Sets the color of the object.
- ✨ `isFilled()` : Returns whether the object is filled.
- ✨ `setFilled(boolean filled)` : Sets the filled status.
- ✨ `getDateCreated()` : Returns the creation date.
- ✨ `toString()` : Returns a string representation of the object.

👉 Subclasses: `Circle`, `Rectangle`

✨ Circle Class

✨ Inherits from `GeometricObject` .

✨ Adds property: `radius` .

✨ Adds methods: `getRadius()` , `setRadius()` , `getArea()` , `getPerimeter()` ,
`getDiameter()` .

✨ Rectangle Class

✨ Inherits from `GeometricObject` .

✨ Adds properties: `width` , `height` .

✨ Adds methods: `getWidth()` , `setWidth()` , `getHeight()` , `setHeight()` , `getArea()` ,
`getPerimeter()` .

11.3. Using the `super` Keyword

Motivation

✨ In OOP, when a subclass has a method or property with the same name as its superclass, it can cause confusion about which version to use. This is known as **shadowing** or **hiding** problem.

✨ Therefore, Java provides the `super` keyword to resolve this issue.

👉 Calling Superclass Constructors

✨ The `super` keyword can be used to call the constructor of the superclass from the subclass constructor.

Syntax:

```
super.attributeName; // Access superclass attribute  
super.methodName(args); // Call superclass method  
super(args); // Call superclass constructor
```

Explain:

✨ The `super` keyword is used to call attributes, methods, or constructors of the superclass.

Example: Calling Superclass Attribute

```
class Animal {  
    String name = "Animal";  
}  
  
class Dog extends Animal {  
    String name = "Dog";  
  
    void printNames() {  
        System.out.println(name); // prints "Dog"  
        System.out.println(super.name); // prints "Animal"  
    }  
}
```

Example: Calling Superclass Constructor

```
class Animal {  
    String name;  
    Animal(String name) { this.name = name; }  
}  
class Dog extends Animal {  
    Dog(String name) { super(name); } // Calls Animal constructor  
    void printName() { System.out.println(name); }  
}
```


Example: Calling Superclass Method

```
class Animal {
    String name;
    Animal(String name) { this.name = name; }
    void printName() { System.out.println("Animal name: " + name); }
}
class Dog extends Animal {
    Dog() { super("Dog"); }
    @Override
    void printName() {
        super.printName(); // Calls Animal's printName method
        System.out.println("Dog name: " + name);
    }
}
```

👉 Constructor Chaining

✨ Superclass Constructor Invocation:

- ✨ When a subclass object is created, its constructor automatically calls the constructor of its immediate superclass.
- ✨ This process continues up the inheritance hierarchy until the `Object` class constructor is executed.
- ✨ Ensures that all inherited fields are properly initialized before the subclass's own initialization.

✨ Constructor Chaining:

- ✨ Guarantees that constructors for all superclasses are executed in the correct order.
- ✨ Achieved using the `super()` call as the first statement in a subclass constructor.
- ✨ If not explicitly called, Java inserts a default `super()` call.

Example: Chain of Inheritance

```
class Object {  
    Object() { System.out.println("Object constructor called"); }  
}  
class Animal extends Object {  
    Animal() { System.out.println("Animal constructor called"); }  
}  
class Dog extends Animal {  
    Dog() { System.out.println("Dog constructor called"); }  
}
```

Explain: When a `Dog` object is created, `Dog` 's constructor calls `Animal` 's constructor, `Animal` 's constructor calls `Object` 's constructor, resulting in the following output:

```
Object constructor called  
Animal constructor called  
Dog constructor called
```

Visualization

```
public ClassName() {  
    // some statements  
}
```

Equivalent

```
public ClassName() {  
    super();  
    // some statements  
}
```

```
public ClassName(parameters) {  
    // some statements  
}
```

Equivalent

```
public ClassName(parameters) {  
    super();  
    // some statements  
}
```

11.4. Overriding Method

What is Method Overriding?

- ✨ **Method Overriding** occurs when a subclass provides a specific implementation of a method that is already defined in its superclass.
- ✨ It allows the subclass to modify or extend the behavior of the superclass method.

👉 Method Overriding Rules

- ✨ Same method name, parameters, and (or covariant) return type as superclass.
- ✨ Can throw same or narrower exceptions.
- ✨ Access level: same or more permissive (not more restrictive).
- ✨ Can be marked `final` to prevent further overriding.

Example: Overriding Method

```
class B {  
    public void p(double i) { System.out.println(i * 2); }  
}  
class A extends B {  
    @Override  
    public void p(double i) { System.out.println(i); }  
}
```


👉 Using `@Override` Annotation

- ✨ The `@Override` annotation is used to indicate that a method is intended to override a method in the superclass.
- ✨ It helps catch errors at compile time if the method does not actually override anything.
- ✨ If the method signature does not match any method in the superclass, the compiler will generate an error.
- ✨ It improves code readability and maintainability by clearly indicating overridden methods.

Example: Annotation Usage

```
@Override  
public String toString() {  
    return super.toString() + "\nradius is " + radius;  
}
```

TIP: Customizing the Built-in `toString()` Method

👉 What is the Built-in `toString()` Method?

- ✨ The `toString()` method is a built-in method in Java that returns a string representation of an object.
- ✨ It is defined in the `Object` class, which is the root class of all Java classes.
- ✨ By default, it returns a string that consists of the class name followed by the `@` character and the object's hash code in hexadecimal.

Example:

```
Circle circle = new Circle(5.0, "red", true);  
System.out.println(circle.toString()); // Output: Circle@1a2b3c4
```

Explain: The default `toString()` method does not provide meaningful information about the object's state.

👉 Customize `toString()` Method using `@Override`

✨ Override `toString()` to return a descriptive string representation of the object.

Example: Customizing `toString()` method in Circle Class

```
@Override
public String toString() {
    return "Circle: radius = " + radius + ", color = " + getColor() + ", filled = " + isFilled();
}
```

Usage:

```
System.out.println(object); // Implicitly calls object.toString()
```

Output:

```
Circle: radius = 5.0, color = red, filled = true
```

👉 **Benefits of Overriding toString()**

- ✨ Provides a clear and concise description of the object.
- ✨ Useful for debugging and logging.
- ✨ Simplifies printing object details without needing to call multiple getter methods.

11.5. Overloading Method

What is Method Overloading?

- ✨ **Method Overloading** occurs when multiple methods in the same class (or subclass) have the same name but different parameter lists (different types, number, or order of parameters).
- ✨ It allows methods to perform similar functions with different types or numbers of inputs.
- ✨ Overloading is resolved at compile time (static binding).
- ✨ It is not related to inheritance; it can occur within the same class or across subclasses.

👉 Method Overloading Rules

- ✨ Must differ in parameter list (number, type, or order).
- ✨ Return type alone can't distinguish overloads.
- ✨ Access modifiers can differ.
- ✨ No `@Override` needed.

Example: Overloading Methods

```
class A {  
    public void p(int i) { System.out.println("int: " + i); }  
    public void p(double i) { System.out.println("double: " + i); }  
}
```

👉 Overriding vs. Overloading

Feature	Overriding (Inheritance)	Overloading (Same Class)
Definition	Redefines superclass method in subclass	Same method name, different parameters
Classes	Superclass & subclass	Same or related classes
Signature	Same as superclass	Different parameter lists
Return Type	Same or covariant	Can differ
Access Modifier	Same or more permissive	Can differ
Binding	Dynamic (runtime)	Static (compile-time)
Annotation	<code>@Override</code> (recommended)	Not needed
Purpose	Specialize inherited behavior	Handle different input types/counts
Example	<code>public void method()</code> in subclass	<code>public void method(int x)</code> and <code>public void method(double y)</code> in same class

11.6. Polymorphism

Motivation

- ✨ In OOP, it's common to design methods that can work with different types or numbers of arguments.
- ✨ This flexibility allows code to be more reusable and adaptable to various scenarios.
- ✨ Therefore, the concept of **Polymorphism** is introduced to solve this problem.

👉 What is Polymorphism?

✨ **Poly-** means "many", and **Morph** means "forms".

✨ **Polymorphism** allows objects of different classes to be treated as objects of a common superclass.

✨ Enables a single interface to represent different underlying forms (data types).

✨ There are two main types of polymorphism in Java:

✨ **Compile-time Polymorphism:** Achieved through method overloading.

✨ **Runtime Polymorphism:** Achieved through method overriding and dynamic binding.

👉 Why Use Polymorphism?

- ✨ **Flexibility:** Allows methods to operate on objects of different classes.
- ✨ **Extensibility:** New classes can be added without modifying existing code.
- ✨ **Code Clarity:** Simplifies code by allowing a single method to handle multiple types.
- ✨ **Dynamic Behavior:** The actual method invoked is determined at runtime based on the object's type.

Example: Calculator with Polymorphism

- ✨ A single `add` method can handle different types of inputs (integers, doubles) and different numbers of parameters.

Polymorphism (Method Overloading)

Syntax:

```
class A {  
    void methodName() {  
        // Method implementation  
    }  
    void methodName(int param) {  
        // Overloaded method implementation  
    }  
    void methodName(String param) {  
        // Yet another overloaded method implementation  
    }  
    void methodName(int param1, String param2) {  
        // Another overloaded method implementation  
    }  
}
```


Example: Different forms of `add()` method

```
class Calculator {  
    int add(int a, int b) { return a + b; }  
    int add(int a, int b, int c) { return a + b + c; }  
    double add(double a, double b) { return a + b; }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.add(5, 10));           // 15  
        System.out.println(calc.add(5, 10, 15));       // 30  
        System.out.println(calc.add(5.5, 10.5));       // 16.0  
    }  
}
```

Polymorphism (Method Overriding)

Syntax:

```
class Superclass {  
    void methodName() {  
        // Superclass implementation  
    }  
}  
class Subclass extends Superclass {  
    @Override  
    void methodName() {  
        // Subclass implementation  
    }  
}
```

Polymorphism (Built-in Types)

Example: Using `+` Operator with Different Types

```
System.out.println(5 + 10);           // 15 (addition)
System.out.println("Hello" + "Java"); // "HelloJava" (concatenation)
System.out.println(5 + " apples");    // "5 apples" (concatenation)
```

Polymorphism (User-Defined Types)

Example: Complex Number Addition

```
class Complex {  
    double real, imag;  
    Complex(double r, double i) { real = r; imag = i; }  
    Complex add(Complex c) { return new Complex(real + c.real, imag + c.imag); }  
}  
class Main {  
    public static void main(String[] args) {  
        Complex c1 = new Complex(1.0, 2.0), c2 = new Complex(3.0, 4.0);  
        Complex result = c1.add(c2);  
        System.out.println(result.real + " + " + result.imag + "i");  
    }  
}
```

Polymorphism (Runtime Through Inheritance)

Example: Runtime Polymorphism with Inheritance

```
class Animal { void sound() { System.out.println("Animal makes sound"); } }
class Dog extends Animal { @Override void sound() { System.out.println("Dog barks"); } }
class Cat extends Animal { @Override void sound() { System.out.println("Cat meows"); } }
public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog(), a2 = new Cat();
        a1.sound(); // Dog barks
        a2.sound(); // Cat meows
    }
}
```

End of the Chapter